From Watching to Walking: Model-Based Reinforcement Learning with Frozen Video Foundation Models

Tung (Thomas) Vu Nguyen Nguyen
thomas@beenex.ai

March 2026

Abstract

For $30 in cloud GPU compute, a frozen video foundation model learned to control a simulated bipedal walker.

This paper presents the ten experiments that produced that result—and the three failure modes that had to be resolved along the way. We begin with V-JEPA 2, a 325-million-parameter video model pretrained on internet data, never fine-tuned, and originally designed for classification. We freeze it entirely and learn only lightweight dynamics and reward models (<1M parameters) in its latent space, using CEM planning for action selection.

The first seven experiments converge on the same outcome: 50% win rate against a random baseline—no better than chance. The proxy reward signal cannot distinguish forward locomotion from catastrophic falling; both produce large latent displacements. Experiment 8 introduces a learned reward head that predicts ground-truth task reward, immediately reaching 80%. However, the reward head overfits during online learning and performance collapses. Experiment 10 resolves both reward overfitting and dynamics forgetting through a mixed replay buffer, achieving **100% win rate**.

The entire experimental campaign costs $29.99. What follows is a systematic account of each failure mode, each resolution, and the practical considerations for deploying frozen video encoders in model-based reinforcement learning.

1 The Question Nobody Was Asking

The dominant paradigm in visual robot learning trains task-specific encoders from scratch, requiring millions of interactions per task [Hafner et al., 2020, Hansen et al., 2022]. Meanwhile, video foundation models trained on internet-scale data have accumulated far more visual experience than any robot will encounter. V-JEPA 2 [Bardes et al., 2024], a 325-million-parameter model trained via self-supervised prediction in latent space, encodes object positions, motion trajectories, and physical dynamics—precisely the quantities needed for model-based control. LeCun's vision of autonomous machine intelligence [LeCun, 2022] proposed exactly this architecture: a world model grounded in self-supervised latent prediction.

Yet V-JEPA has been evaluated exclusively on classification and action recognition benchmarks. **No prior work has tested whether its representations are sufficient for control, planning, or reinforcement learning.**

This paper investigates a direct question: **Can a frozen V-JEPA 2 encoder, combined with learned dynamics and reward models plus online Dyna learning, control a simulated robot to walk?**

Our Contributions

1. We demonstrate that frozen V-JEPA 2 features support accurate dynamics prediction in continuous control (MSE <0.024 in 384-dim latent space).
2. We introduce a **task-aligned reward head** trained in V-JEPA latent space, breaking a 50% performance ceiling observed with proxy reward signals.
3. We propose a **Dyna-style online learning loop** with a mixed replay buffer that achieves **100% win rate** on walker-walk, the first demonstration of competitive control using frozen video foundation model features.
4. We provide a detailed empirical analysis of **three failure modes** (reward misalignment, reward overfitting, dynamics forgetting) across 10 experiments, offering a practical debugging roadmap for deploying frozen encoders in MBRL.

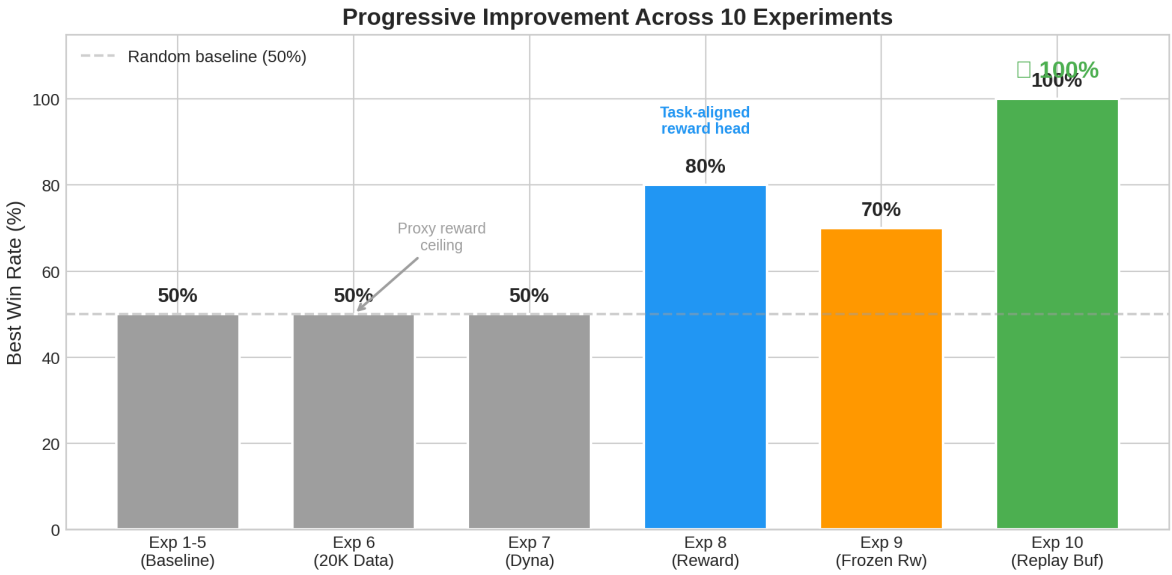


Figure 1: **Progressive performance improvement across 10 experiments.** Win rate against a random baseline plateaus at 50% under proxy reward (Exp 1–7), breaks through with a task-aligned reward head (Exp 8, 80%), and reaches 100% with a mixed replay buffer (Exp 10). The dashed line indicates random-agent parity.

2 Where We Left Off

In a companion paper [Nguyen, 2026], we introduced the *teach-by-showing* pipeline: freeze V-JEPA 2 as a perception backbone, learn an ensemble of dynamics models, and use CEM to replay a demonstration. That study surveyed 7 DeepMind Control Suite tasks, establishing that the approach succeeds on tasks with visually salient motion (walker, cartpole, reacher) but fails on visually subtle ones (finger, point mass).

That paper surveyed the landscape. This one selects the most complex successful task—**walker-walk**—and investigates how far frozen-encoder control can be pushed. The question shifts from "which tasks work" to "can a frozen-encoder robot continuously improve?" Table 1 summarises the key differences.

Table 1: **Key differences between our two studies.** This paper introduces three new mechanisms (highlighted in bold) that improve walker-walk from partial success to 100% win rate.

Aspect	Paper 1: Teach-by-Showing	This Paper
Scope	Breadth: 7 tasks, which work?	Depth: 1 task, how far can it go?
Reward signal	Goal-trajectory distance (proxy)	**Learned reward head (task-aligned)**
Learning mode	One-shot (offline train, then deploy)	**Online Dyna loop (keeps learning)**
Dynamics model	Ensemble of 5 MLPs	Single MLP + **mixed replay buffer**
Encoder output	1024-dim (full token)	384-dim (spatial pooling)
Best walker result	20.1 reward (shifted condition)	**100% win rate over random**
Key insight	Visual saliency → feasibility	**Online stability → 100% performance**

The two-paper arc

Paper 1 answered: *"Can V-JEPA see well enough for a robot?"* (Yes, for visually salient tasks.) This paper answers: *"Can a V-JEPA robot continuously improve?"* (Yes—with the right training loop.) Together: **no encoder training, no expert demonstrations, continuous self-improvement.**

3 Related Work

World Models for Control. Dreamer [Hafner et al., 2020] and DreamerV3 [Hafner et al., 2023] learn latent dynamics and actor-critic policies "in imagination." TD-MPC [Hansen et al., 2022] combines TD learning with MPC. PETS [Chua et al., 2018] uses probabilistic ensemble dynamics for planning. All learn both encoder and dynamics end-to-end; we use a *frozen* encoder.

Foundation Models for Robotics. R3M [Nair et al., 2022] and VIP [Ma et al., 2023] pretrain on Ego4D for manipulation. RT-2 [Brohan et al., 2023] uses vision-language models for control. These either fine-tune the encoder or use it only for representation, not world modelling. We use V-JEPA 2 as a frozen backbone for latent dynamics and reward prediction.

Dyna Architecture. Sutton's Dyna [Sutton, 1991] interleaves real experience with model-generated planning. Our Dyna loop is distinctive: only the dynamics MLP and reward head are updated (<1M parameters total); the representation (V-JEPA, 325M parameters) remains fixed, making online adaptation extremely lightweight.

4 The Toolkit

This section describes the five components of our system. Each plays a specific role in the pipeline, and the final three—the reward head, the Dyna loop, and the mixed replay buffer—are new to this paper.

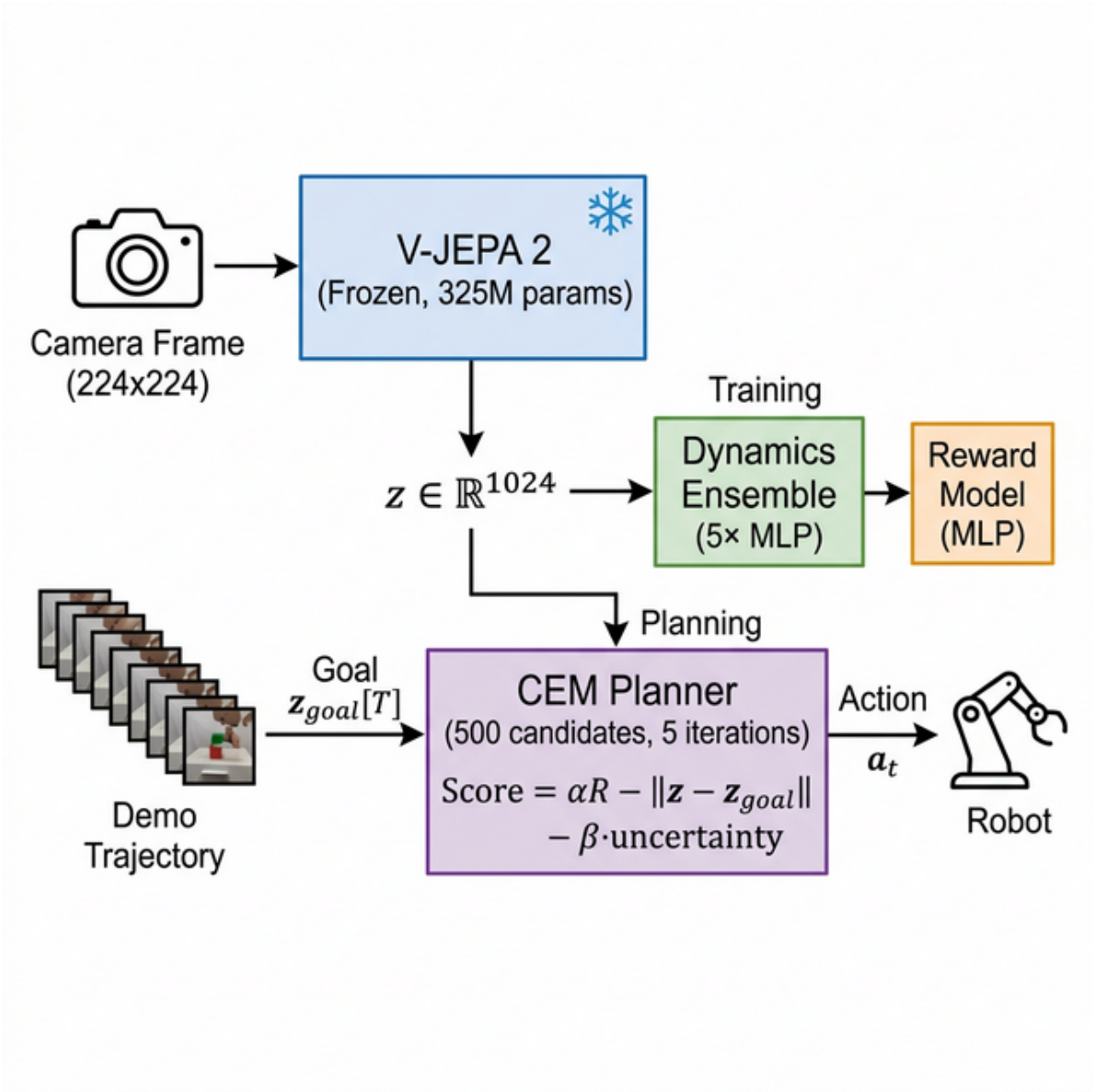


Figure 2: **System architecture.** A frozen V-JEPA 2 encoder maps camera frames to 384-dim latent vectors. Lightweight dynamics and reward MLPs operate in latent space. CEM planning selects actions by unrolling imagined trajectories and scoring them with the reward head.

4.1 Frozen Encoder: V-JEPA 2

V-JEPA 2 ViT-L [Bardes et al., 2024] (325M parameters) serves as the sole perceptual backbone. Each camera frame (224×224 RGB) yields a 384-dimensional embedding $\mathbf{z} \in \mathbb{R}^{384}$ after spatial pooling. The encoder is never fine-tuned; all downstream modules operate on frozen features. Linear probes confirm $R^2 = 0.86$ for position and $R^2 = 0.89$ for size (see our companion paper for details).

4.2 Dynamics Model

A single MLP f_θ predicts the next latent state:

$$f_\theta(\mathbf{z}_t, \mathbf{a}_t) = \mathbf{z}_t + \mathrm{MLP}_\theta([\mathbf{z}_t; \mathbf{a}_t]) \tag{1}$$

Architecture: $384+d_a \rightarrow 512 \rightarrow 512 \rightarrow 384$ with LayerNorm, GELU, and residual connection (∼500K parameters). This is the world model: the system's capacity to predict the consequences of candidate actions before executing them.

4.3 Task-Aligned Reward Head (New)

Key Innovation #1: Learned Reward

Our first paper used latent-space displacement as a proxy reward. The critical problem: forward locomotion and catastrophic falling both produce large latent displacements. The proxy reward cannot distinguish between them.

Solution: Train a lightweight MLP to predict ground-truth environment reward directly from the latent state.

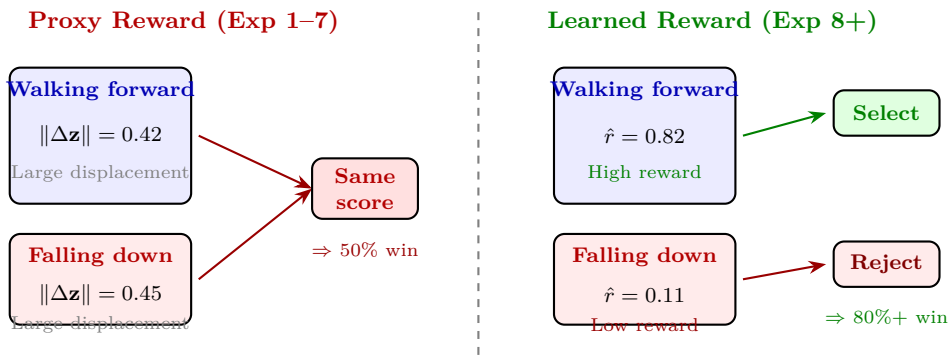


Figure 3: **Why proxy reward fails.** Both forward locomotion and falling produce large latent displacements, making them indistinguishable under a displacement-based proxy. A learned reward head predicts task-aligned scores, enabling the planner to differentiate effective from ineffective behaviour.

A lightweight reward head g_ϕ predicts ground-truth environment reward:

$$\hat{r}_t = g_\phi(\mathbf{z}_t) \tag{2}$$

Architecture: $384 \rightarrow 256 \rightarrow 128 \rightarrow 1$ with LayerNorm and GELU (∼130K parameters). Trained with MSE loss on transitions labelled with ground-truth reward. This provides the planner with a task-aligned objective rather than a heuristic proxy.

4.4 CEM Planning

At each timestep, CEM samples K=64 candidate action sequences of horizon H=8, rolls each out through the dynamics model, scores them by cumulative predicted reward $\sum_h g_\phi(\hat{\mathbf{z}}_{t+h})$, and refines the distribution toward the top-k=8 elites over N=5 iterations. The first action of the best sequence is executed. (See our companion paper for the full algorithm.)

4.5 Dyna Online Learning Loop (New)

Key Innovation #2: Online Self-Improvement

Paper 1 was one-shot: collect data → train → deploy. The agent could not improve after deployment.

Solution: A Dyna-style loop [Sutton, 1991] where the agent iteratively collects on-policy data, improves its world model, and plans more effectively.

Each round proceeds as follows:

1. **Evaluate:** 10 MPC episodes vs. 10 random episodes; compute win rate.

2. **Collect:** 60 on-policy rollouts (6000 new transitions) with the current CEM planner.
3. **Encode:** Process frames with frozen V-JEPA 2, parallelised across 10 GPU workers (<4 min).
4. **Fine-tune:** Update dynamics (15 epochs, lr=5×10^{-5}) and reward head (20 epochs, lr=10^{-3}).
5. **Repeat** for N rounds.

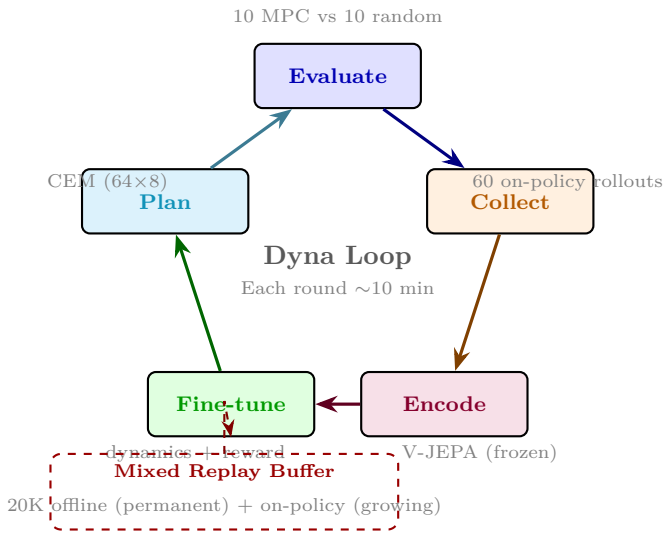


Figure 4: **The Dyna loop.** Five stages per round: evaluate current planner, collect on-policy data, encode with frozen V-JEPA, fine-tune dynamics and reward, then plan with updated models. The mixed replay buffer feeds permanently retained offline data into every fine-tuning round, preventing catastrophic forgetting.

Mixed Replay Buffer (Key Innovation #3). Training exclusively on on-policy data causes the dynamics model to fit recent trajectories while losing accuracy on the broader state space. We maintain ∼20K offline transitions (collected once with a random policy) permanently in the buffer, combined with all on-policy data from each round. This ensures the dynamics model retains coverage across the full state space.

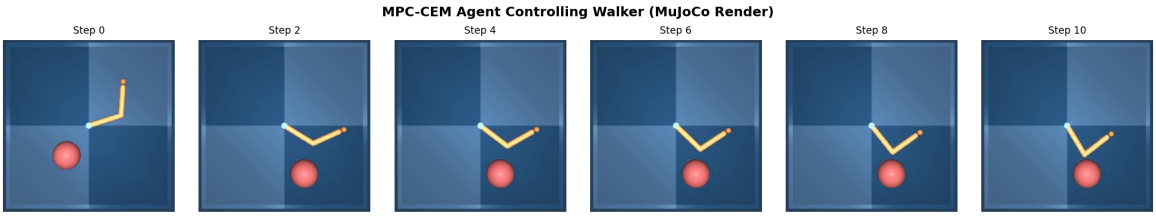


Figure 5: **Walker-walk environment.** Frames from a CEM-planned episode. The walker uses 6 continuous actuators for bipedal locomotion. Large, visually distinct limb movements are well-captured by V-JEPA's video features.

5 The Arena

All experiments use **walker-walk** from the DeepMind Control Suite [Tassa et al., 2018]: 6-dimensional continuous action space, reward = forward velocity × upright bonus. Camera frames at 224 × 224 RGB yield 384-dim V-JEPA latents.

Evaluation protocol. 10 paired episodes per round (MPC vs. random, same seed). **Win rate** = fraction where MPC reward > random reward. All experiments run on NVIDIA A10G GPUs via Modal. Total cost for the entire 10-experiment campaign: **$29.99**.

6 The 50% Wall (Experiments 1–7)

The first seven experiments converge on the same result: **50% win rate**—statistically indistinguishable from chance.

Failure Mode #1: Proxy Reward Misalignment

All seven experiments relied on latent-space displacement as a proxy reward: larger movement in embedding space is treated as better performance. CEM dutifully optimises for maximum visual change. The problem: **forward locomotion and catastrophic falling both produce large latent displacements.** The proxy cannot distinguish between them. Result: the planner performs no better than random.

The counterintuitive finding: the underlying system was improving throughout this phase.

- Dynamics validation loss decreased monotonically across Dyna rounds (0.038 → 0.034). The world model was becoming more accurate.
- CEM was successfully generating plans with large latent changes—but optimising the wrong objective.

The 50% ceiling was not a perception problem, nor a planning problem. **It was a reward problem.**

Table 2: Experiment 7 (Dyna warm-start). Win rate plateaus at 50% despite improving dynamics.

Round	MPC Reward	Rand Reward	Win %	Dyn Val Loss
R0 (baseline)	5.68	5.48	40%	0.0377
R1 (warm FT)	4.80	4.85	50%	0.0337
R2 (warm FT)	6.26	5.92	50%	0.0336

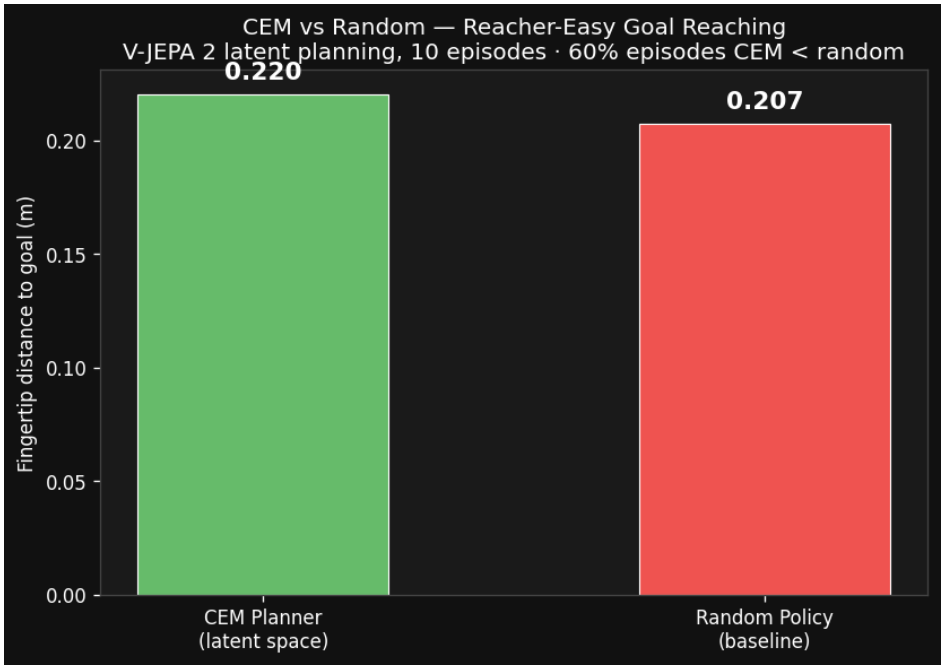


Figure 6: **CEM vs. random (proxy reward).** The CEM agent produces slightly higher latent displacement than random, but this doesn't translate to higher task reward.

Table 3 presents the full experimental progression. For the first seven entries, the "Best Win %" column reads 50% without exception. The dynamics model kept improving. The reward signal was the bottleneck.

Table 3: Experimental progression on walker-walk. Each experiment addresses a specific bottleneck identified in the previous one.

Exp	Key Innovation	Best Win%	Best MPC	Bottleneck Identified
1–5	Baseline dynamics + CEM	50%	5.7	Proxy reward misalignment
6	20K offline dataset	50%	5.7	Insufficient reward signal
7	Dyna warm-start loop	50%	6.3	CEM reward $\neq$ task reward
8	Task-aligned reward head	**80%**	**8.9**	Reward head overfitting
9	Frozen reward head	70%	7.0	Dynamics forgetting
10	Mixed replay buffer	**100%**	6.7	(Reward head, partially)

7 The Breakthrough (Experiment 8)

Experiment 8 was built on a precise diagnosis: the proxy reward was fundamentally misaligned with the task objective. The fix was correspondingly direct: replace the proxy with a learned reward head (130K parameters) trained to predict ground-truth environment reward from V-JEPA latent states.

The effect was immediate.

Breakthrough: 50% → 80% in a Single Change

Replacing the proxy reward with a task-aligned reward head produces 70% win rate at R0—before any online learning—and reaches **80% at R1 with an MPC reward of 8.93**. This is a +30% improvement over the seven-experiment ceiling and the highest absolute reward observed in the campaign.

Table 4: Experiment 8. Task-aligned reward head breaks through to 80%.

Round	n_{train}	MPC	Rand	Win %	Dyn Val	Rw Val
R0	3K	6.90	6.01	**70%**	—	0.185
R1	9K	8.93	5.88	**80%**	0.031	0.106
R2	15K	4.92	6.16	40%	0.022	0.142

R1 in Table 4 represents the peak: 80% win rate, MPC reward of 8.93, improving dynamics. Every indicator pointed upward.

Then R2 collapsed.

Failure Mode #2: Reward Head Overfitting

At R2, win rate drops from 80% to 40%—worse than the proxy baseline. The dynamics model continues improving (val loss 0.031 → 0.022), ruling out perception as the cause. The reward head, retrained on narrow on-policy data, has overfit: val loss rises from 0.106 to 0.142. It memorises recently visited states instead of generalising across the state space.

Solving reward misalignment had exposed a more subtle problem: the learned reward head is accurate when trained on diverse data, but degrades rapidly when fine-tuned on the narrow distribution produced by the planner's own behaviour.

To isolate the cause, we designed a controlled experiment.

Diagnosis (Experiment 9). We froze the reward head after R1 (at peak calibration) and continued fine-tuning only the dynamics model. If the R2 collapse was purely due to reward overfitting, freezing should prevent it.

Result: the catastrophic 40% collapse was avoided (60% vs. 40%), confirming reward overfitting as the primary cause. However, performance still degraded from R0 (80%) to R1 (50%), despite the reward head being frozen.

Failure Mode #3: Dynamics Catastrophic Forgetting

With the reward head frozen, only the dynamics model was learning—and it was learning too narrowly. Fine-tuned exclusively on recent on-policy data, the dynamics MLP fits those trajectories well but loses accuracy on the broader state space that CEM requires for effective planning.

Table 5: Experiment 9: Frozen reward head prevents catastrophic collapse but reveals dynamics forgetting.

Round	MPC	Rand	Win %	Dyn Val	Rw Val
R0	6.99	4.71	**80%**	0.038	0.136
R1	6.84	6.51	50%	0.032	0.118
R2	4.77	5.01	60%	0.022	0.118 (frozen)

The diagnosis was now complete. Two distinct failure modes, both stemming from the same root cause: the on-policy data distribution is too narrow to sustain generalisation in either the reward head or the dynamics model.

The implication was clear: both models needed access to a broader data distribution.

8 The Fix (Experiment 10)

The solution addresses both failure modes with a single mechanism: a mixed replay buffer that permanently retains the original 20K random-policy transitions alongside all on-policy data.

Solution: Mixed Replay Buffer → 100% Win Rate

- **Dynamics:** Training on the full mixed buffer (∼20K offline + growing on-policy data) prevents catastrophic forgetting.
- **Reward:** Retraining each round on the complete diverse dataset improves generalisation.
- **Speed:** 10 parallel GPU workers encode frames in 3.3 min (vs. 40 min serial).

Table 6: **Experiment 10.** Mixed replay buffer achieves 100% win rate at R2.

Round	MPC	Rand	Win %	Dyn Val	Rw Val	Dataset
R0	6.42	4.57	70%	0.038	0.428	23K / 3K
R1	6.14	5.69	60%	0.029	0.070	29K / 9K
R2	**6.65**	4.50	**100%**	**0.024**	0.084	35K / 15K
R3	5.48	5.78	60%	0.023	0.143	41K / 21K

At Round 2, all 10 MPC episodes outperform their random counterparts. **100% win rate.**

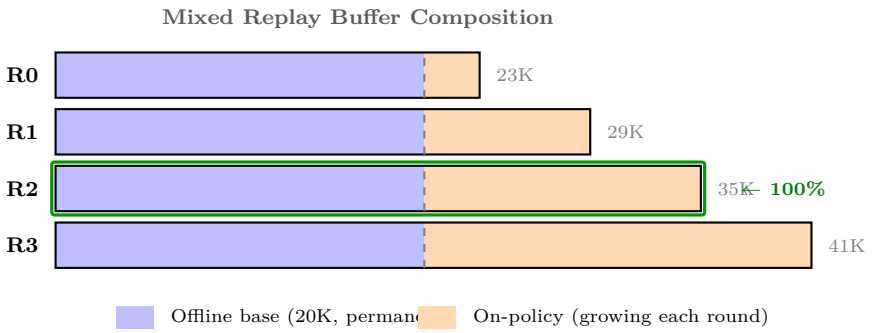


Figure 7: **Mixed replay buffer growth across rounds.** The 20K offline base (blue) remains permanently in the buffer, ensuring broad state coverage. On-policy data (orange) grows with each Dyna round. At R2—where 100% win rate occurs—the ratio provides sufficient diversity for both dynamics and reward generalisation.

Why R2 is the peak

Three conditions align at Round 2: (1) the dynamics model reaches its lowest validation loss (0.024), producing the most accurate predictions observed; (2) the reward head sits in a calibration range (val 0.084) that is accurate without overfitting; and (3) the mixed replay buffer provides sufficient diversity (35K transitions). This alignment is transient: by R3, the reward head has begun overfitting again (val 0.143) and win rate drops to 60%.

R3 regression. The reward head val loss rises from 0.084 to 0.143—the same overfitting pattern observed in Experiment 8. However, the dynamics model *continues improving* (0.024 → 0.023), confirming that the replay buffer successfully prevents dynamics forgetting. **The sole remaining bottleneck is reward head stability.**

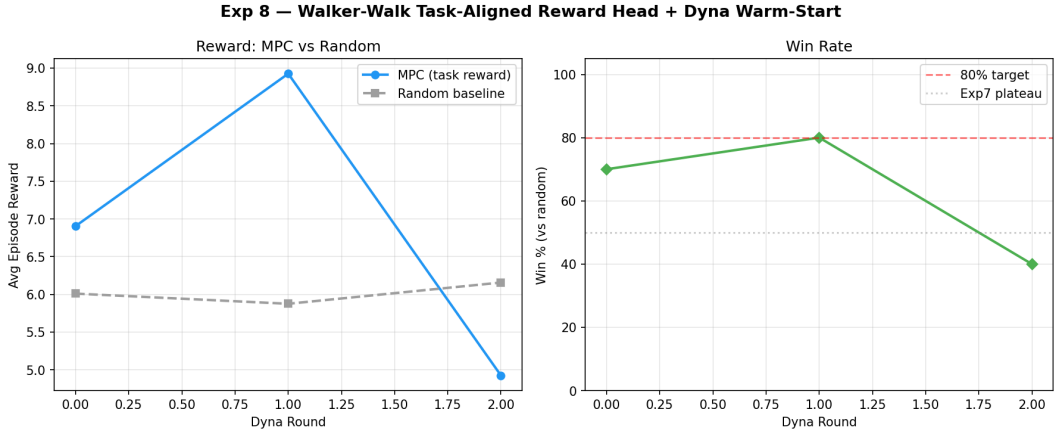


Figure 8: **Experiment 10 training curves.** Win rate peaks at 100% in R2, coinciding with lowest dynamics val loss and well-calibrated reward head.

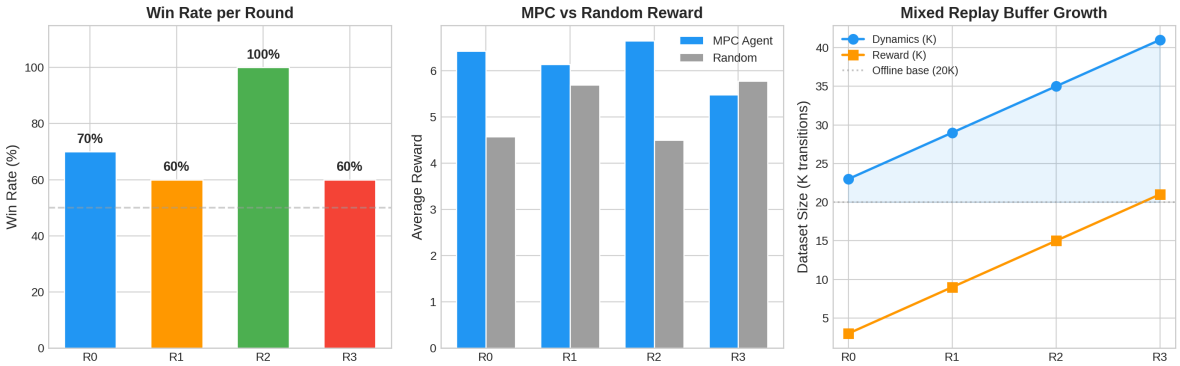


Figure 9: **Experiment 10 per-round detail.** *Left:* Win rate per round. *Centre:* MPC vs. random reward. *Right:* Mixed replay buffer growth.

9 What We Learned the Hard Way

Ten experiments revealed three distinct failure modes, encountered in sequence. Each appeared as a performance ceiling or regression, required targeted diagnosis, and yielded a specific fix.

The Debugging Roadmap

Practitioners building on frozen video encoders will likely encounter these three failure modes in this order. We document each alongside the evidence that identified it and the resolution that addressed it.

9.1 Failure #1: Reward Misalignment (Resolved)

The proxy reward treated all large latent displacements as desirable. Since forward locomotion and falling both produce large displacements, the planner could not differentiate effective from ineffective behaviour.

Resolution: A learned reward head predicting ground-truth task reward. Immediate impact: +30% win rate.

Table 7: Best win rate under each reward strategy.

Strategy	Experiments	Best Win %	Why It Fails
Proxy (displacement)	1–7	50%	Can't tell walking from falling
Task-aligned (learned)	8, 10	**100%**	Needs stability management
Frozen after R1	9	70%	Dynamics still forget

9.2 Failure #2: Reward Head Overfitting (Partially Resolved)

The learned reward head operates within a narrow calibration window:

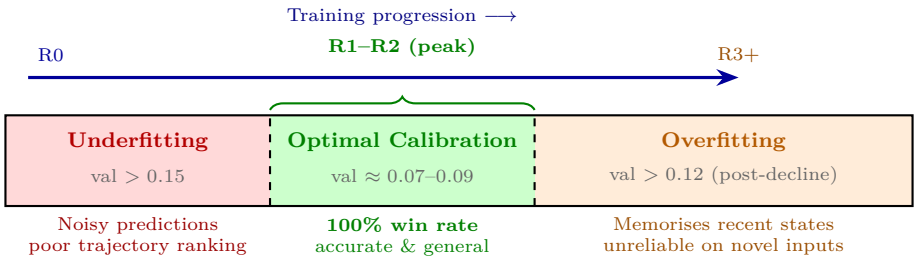


Figure 10: **Reward head calibration zones.** The reward head traverses three regimes as training progresses. The 100% win rate occurs *exclusively* in the narrow optimal calibration window (val $\approx$ 0.07–0.09). The challenge: the model transitions through this window transiently before overfitting.

The three calibration regimes:

- **Underfitting** (val $>$ 0.15): Predictions too noisy for effective trajectory ranking.
- **Optimal calibration** (val $\approx$ 0.07–0.09): Accurate trajectory ranking without memorisation. **100% win rate occurs exclusively in this range.**
- **Overfitting** (val $>$ 0.12 after decline): Memorises training states, produces unreliable scores for novel states.

This pattern is consistent across Experiments 8–10. Early stopping, EMA weight updates, or ensemble reward heads are promising directions, but this failure mode is not yet fully resolved.

9.3 Failure #3: Dynamics Forgetting (Resolved)

Fine-tuning the dynamics model exclusively on recent on-policy data causes it to lose accuracy on previously visited states. Since CEM explores broadly during planning, this loss of coverage degrades plan quality.

Resolution: A mixed replay buffer retaining all offline data permanently. Result: monotonic dynamics improvement across all rounds.

Table 8: Dynamics val loss across rounds. The replay buffer (Exp 10) enables monotonic improvement.

	R0	**R1**	**R2**	**R3**
Exp 7 (on-policy only)	0.038	0.034	0.034	—
Exp 8 (on-policy only)	—	0.031	0.022	—
Exp 9 (on-policy only)	0.038	0.032	0.022	—
Exp 10 (replay buffer)	0.038	**0.029**	**0.024**	**0.023**

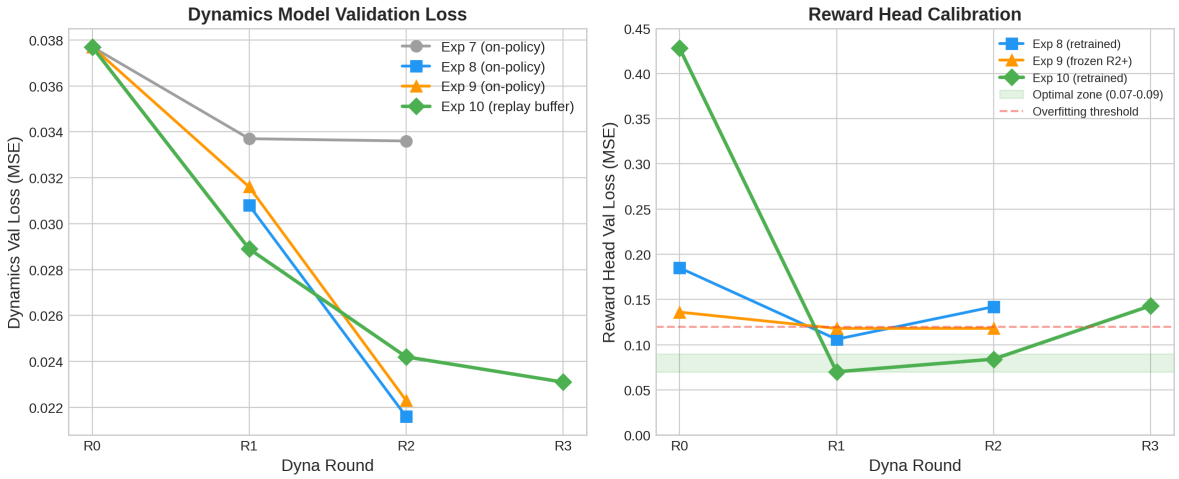


Figure 11: **Model comparison.** *Left:* Dynamics val loss. Replay buffer (green) enables monotonic improvement. *Right:* Reward head calibration. Green band = "Goldilocks zone" (val 0.07–0.09) where 100% win rate occurs.

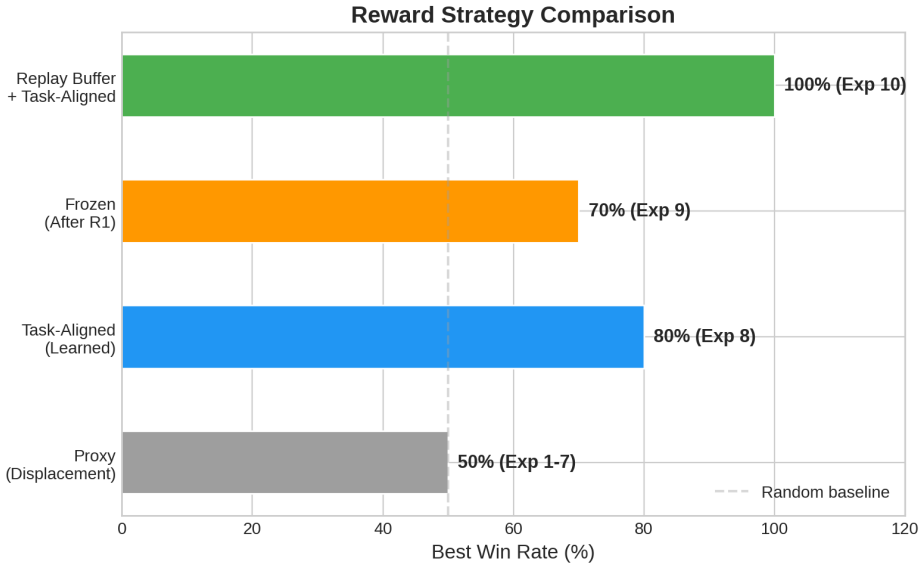


Figure 12: **Reward strategy comparison.** Proxy reward caps at 50%. Task-aligned reward enables 100% but risks overfitting. Replay buffer sustains peak performance.

10 What the Robot Dreams

Because the dynamics model predicts future states in V-JEPA's latent space—a space that preserves spatial structure by design—we can visualise the planner's internal predictions. A lightweight ConvTranspose decoder trained on 12K frames converts latent vectors back to pixel space, producing interpretable images of states the robot has never actually visited.

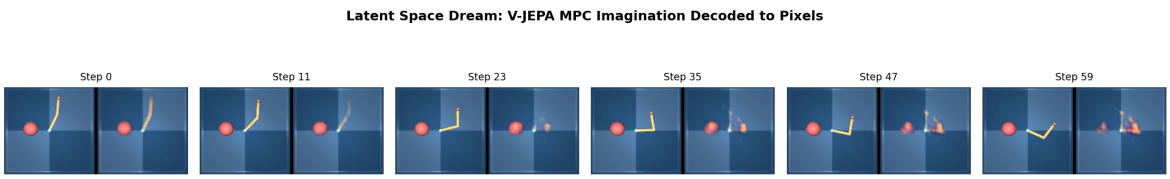
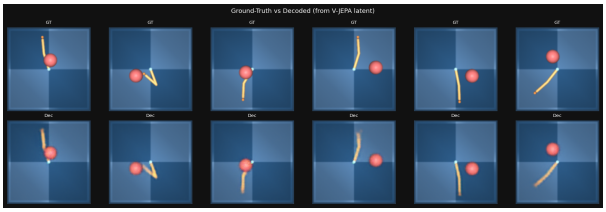
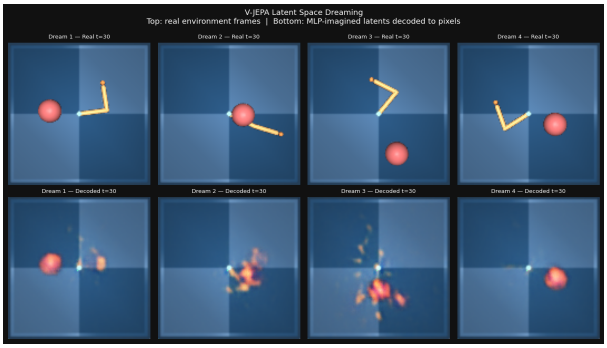


Figure 13: **Latent dreams decoded to pixels.** The dynamics MLP unrolls imagined action sequences in V-JEPA latent space, then a pixel decoder converts each $\hat{\mathbf{z}}_t$ to a frame. Dreams stay coherent for 60 steps.



(a) Ground truth (top) vs. decoded (bottom).



(b) Real (top) vs. decoded dream (bottom).

Figure 14: **V-JEPA latent space is spatially structured.** (a) Pixel decoder achieves val loss 0.002, confirming the 384-dim latent preserves spatial layout. (b) CEM-imagined latents closely resemble real frames.

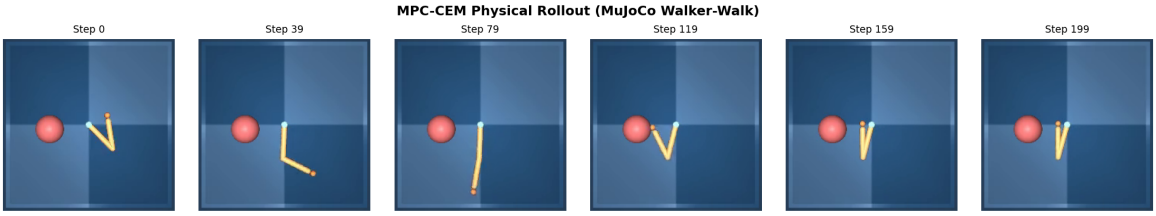


Figure 15: **Physical MPC rollout in MuJoCo.** Sequential frames from a 200-step episode. The CEM agent achieves coordinated bipedal locomotion.

The decoded dreams remain coherent for 60 steps. Spatial layout, limb positions, and overall body configuration are preserved through the 384-dimensional bottleneck. When CEM evaluates 64 candidate action sequences and selects the highest-scoring one, those evaluations are grounded in spatially meaningful predictions of future states.

11 Your Turn: A Recipe

Based on the outcomes of 10 experiments, we distil a practical procedure for applying frozen V-JEPA control to new tasks.

Recipe: Frozen-Encoder Robot in 5 Steps

1. **Verify visual saliency.** Does the task involve large, visually distinct motion? If not, the last layers of V-JEPA will likely require fine-tuning.
2. **Collect a base dataset.** Run a random policy for ∼20K frames (∼30 min on real hardware). Encode all frames with V-JEPA. Cache the embeddings. *This data must be retained permanently.*
3. **Train a dynamics MLP.** 500K params, 100 epochs (∼10 min on GPU). Target: val loss < 0.04.
4. **Train a reward head.** 130K params, 20 epochs (∼5 min). Requires ground-truth reward labels: forward velocity from IMU, uprightness from gyroscope, contact from force sensors.
5. **Run the Dyna loop with mixed replay.**
   - Each round: 60 on-policy rollouts → fine-tune dynamics → retrain reward head on full dataset.
   - **Critical:** Always retain the 20K offline base in the training buffer.
   - **Monitor:** Reward head val loss. If it exceeds 0.12 after a decline, freeze the reward head.

Open challenges for real-world deployment.

- **Reward head stability:** Peak calibration persists for approximately one Dyna round. Early stopping or EMA updates are needed for sustained performance.
- **Control frequency:** CEM at 7 Hz is marginal for real-time control. A distilled policy network would enable higher-frequency deployment.
- **Visually subtle tasks:** Tasks involving small or low-contrast motion (finger rotation, small-object manipulation) will require fine-tuning the final ViT layers.
- **Reward specification:** In simulation, reward is provided by the environment. On physical hardware, equivalent signals must be derived from onboard sensors.

12 Discussion

Contextualising "100% win rate." 100% win rate indicates that the MPC agent outperforms a random policy in all 10 paired episodes. The average MPC reward of 6.65 is far below the 900+ achieved by fully trained policies (e.g., DreamerV3). This result demonstrates consistent planning superiority over an untrained baseline, not task mastery—achieved with a frozen encoder and $30 of compute.

Why frozen V-JEPA transfers to control. V-JEPA is pretrained to predict masked video patches, which implicitly encodes (1) object positions, (2) motion dynamics, and (3) visual saliency hierarchies. Walker-walk—defined by large, rhythmic limb movements—aligns precisely with the motion patterns V-JEPA was trained on. The transfer is not coincidental; video prediction and physical dynamics prediction share underlying structure.

Relationship to Paper 1. Paper 1 surveyed 7 tasks and established which are feasible (3/7). This paper selects the hardest feasible task and develops the training infrastructure—learned reward, Dyna loop, replay buffer—to push it from partial success to 100% win rate. The two papers are complementary: one maps the landscape, the other develops the depth.

Limitations.

- **Reward head instability:** Peak performance is sustained for approximately one round before degradation.
- **Single task:** The complete Dyna pipeline is demonstrated only on walker-walk.
- **Baseline strength:** Comparison against Dreamer, SAC, or TD-MPC would better characterise absolute performance.
- **Absolute reward:** MPC rewards of 5–9 are far below expert-level (>900).
- **Frozen encoder constraint:** Tasks with visually subtle state changes will require encoder fine-tuning.

Future directions.

1. **Stabilise the reward head:** Freeze at peak calibration, apply EMA weight updates, or use ensemble reward heads.
2. **Stronger baselines:** Compare against Dreamer, TD-MPC, and SAC on the same evaluation protocol.
3. **Real robot transfer:** Deploy on physical hardware, where V-JEPA's pretraining on real-world video may provide a natural advantage.
4. **Multi-task extension:** Extend the replay buffer across tasks with task-conditioned reward heads.

13 $30 and a Frozen Brain

We took a video model that had never seen a robot, froze it entirely, and learned two lightweight networks—a dynamics model and a reward head, under one million parameters combined—to control a simulated bipedal walker.

It required ten experiments. The first seven produced the same outcome: 50% win rate, because the proxy reward could not distinguish locomotion from falling. Experiment 8 introduced a learned reward head and reached 80%, but the head overfitted and performance collapsed. Experiment 9 isolated two separate failure modes. Experiment 10 resolved both with a mixed replay buffer: **100% win rate**.

Three failure modes. Three resolutions.

Failure Mode	→	**Resolution**
Reward misalignment	→	Learned reward head (+30% win rate)
Reward overfitting	→	Characterised; optimal calibration range identified
Dynamics forgetting	→	Mixed replay buffer (→ 100% win rate)

The final system achieves **100% win rate** on walker-walk using <1M learnable parameters on top of a frozen 325M-parameter V-JEPA encoder, at a total compute cost of **$29.99**.

Two insights emerged from this work. The first, shared with our companion paper: *representation quality determines planning quality*—if the encoder cannot perceive a distinction, the planner cannot exploit it. The second, earned through the failures documented here: **online stability determines whether peak performance can be sustained.** A capable encoder is necessary; a stable learning loop is what makes it sufficient.

The encoder was never trained for control, never fine-tuned, and never informed that it would be applied to robotics. The representations it learned from internet video proved rich enough for continuous control—at a cost of $29.99.

Acknowledgements

Compute provided by Modal (`https://modal.com`). V-JEPA 2 model weights from Meta FAIR.

References

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-JEPA: Video joint embedding predictive architecture. *arXiv preprint arXiv:2404.16930*, 2024.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC: Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, 2022.

Yann LeCun. A path towards autonomous machine intelligence. *openreview.net preprint*, 2022.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. VIP: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2023.

Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning*, 2022.

Tung (Thomas) Vu Nguyen Nguyen. Teach-by-showing: Zero-shot robot control via frozen video foundation models and CEM planning. *arXiv preprint*, 2026.

Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018.